



SMARTBOOK

Project Documentation

A modern Accounting Management System for small and medium-sized businesses



Contents

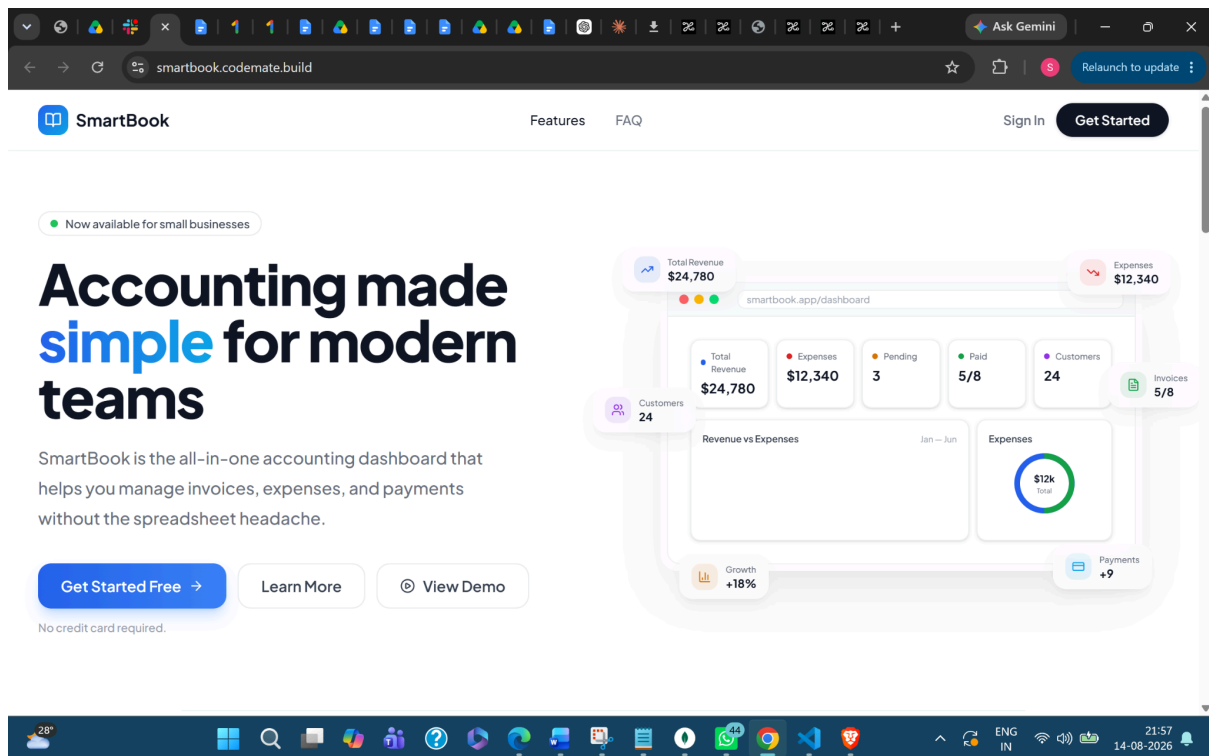
- 01** Project at a Glance
 - 02** Problem, Users & Product Value
 - 03** Customer Persona
 - 04** Architecture & Technology
 - 05** Core Accounting Workflow
 - 06** Feature Map
 - 07** Build: Design Mode
 - 08** Build: Code Mode
 - 09** Build Prompt Highlights
 - 10** CORA: Planning & Implementation
 - 11** CORA Task Archive
 - 12** Implementation Milestones
 - 13** Backend, Data & Integrations
 - 14** Authentication & User Management
 - 15** Responsive UI & Polish
-

SMARTBOOK

01. Project at a Glance

SmartBook is a web-based accounting management system designed for small and medium-sized businesses.

It brings customers, vendors, invoices, expenses, payments, reports, and company management into one centralized platform.



02. Problem, Users & Product Value

Problem

Small businesses often manage customers, invoices, expenses and payments across different tools. SmartBook brings these operations together in one system.

Users

Administrator

- Manages company, users, customers, vendors, invoices and reports.

Staff / Accountant

- Creates invoices, manages expenses, records payments and views customers.

Product Value

- Centralized accounting workflow
 - Easy customer and vendor management
 - Invoice and payment tracking
 - Expense management
 - Financial reporting
 - Role-based access
-

03. Customer Persona

Primary Persona: Small Business Owner / Administrator

Goal: Manage the company's complete financial activity from one place.

Needs:

- Quick overview of business finances
- Customer and vendor management
- Invoice creation and tracking
- Expense and payment tracking
- Financial reports

Pain Points:

- Managing accounting data across multiple tools
- Difficulty tracking pending payments
- Lack of centralized financial information

SmartBook Solution:

Provides one centralized dashboard where the administrator can manage and monitor the company's accounting operations.

04. ARCHITECTURE & TECHNOLOGY

SmartBook uses a frontend-backend architecture for managing accounting operations.

Frontend: React.js

Styling: Tailwind CSS

Routing: React Router

Backend: Node.js + Express.js

Database: MongoDB

Authentication: Custom JWT Authentication

Architecture

User → React.js → Express.js API → Node.js → MongoDB

05. CORE ACCOUNTING WORKFLOW

The main SmartBook workflow connects the core accounting operations in one platform.

Authentication → Dashboard → Customers / Vendors → Invoices → Payments → Expenses → Reports → Settings

Users can manage customers, vendors, invoices, payments and expenses while keeping their accounting data connected in one system.

06. FEATURE MAP

Dashboard

Overview of revenue, expenses, pending payments, invoices and recent transactions.

Customers

Manage customer details, contact information and customer records.

Vendors

Manage vendor information and business-related records.

Invoices

Create, manage and track customer invoices.

Payments

Record and track payments received from customers.

Expenses

Record and manage business expenses.

Reports

View financial reports and accounting insights.

Company Profile

Manage company information and business details.

Settings

Manage application and account configuration.

07. Build: Design Mode

Objective

Create a professional accounting SaaS interface for SmartBook with a clean, responsive and enterprise-style design.

Design Direction

- Minimal interface
- White and gray surfaces
- Blue primary accent
- Responsive layout
- Dashboard analytics
- Collapsible sidebar
- Professional typography
- No unnecessary visual effects

The Design Mode prompt specifically defined the dashboard, sidebar, accounting modules, charts and visual direction.

Prompts Used

Prompt 1 : SmartBook UI Design

Design a modern, production-ready web application UI for an accounting software called “SmartBook”. The design should feel like a real SaaS product built by a professional product company rather than an AI-generated concept. Create a clean, minimal and professional accounting experience for small and medium-sized businesses, with a white background, subtle gray surfaces, blue primary accent, rounded components, soft shadows and spacious layouts.

Prompt 2 : Dashboard & Navigation

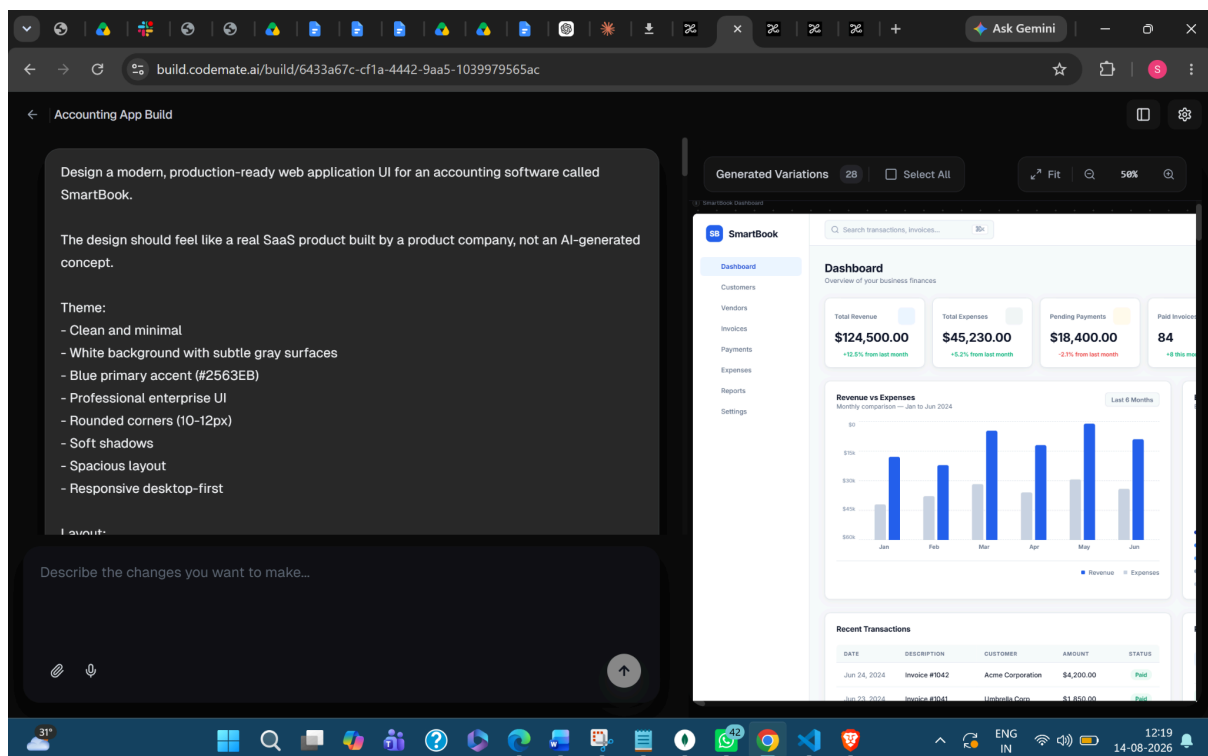
Design the SmartBook application with a responsive desktop-first layout and a collapsible left sidebar. Include Dashboard, Customers, Vendors, Invoices, Payments, Expenses, Reports and Settings in the main navigation. The dashboard should provide a clear overview of total revenue, total expenses, pending payments, paid invoices and active customers, along with revenue and expense charts and a recent transactions section. Keep the interface professional, structured and easy to scan.

Prompt 3 : Accounting Modules

Extend the SmartBook interface into a complete accounting management experience. Create consistent pages for Customers, Vendors, Invoices, Payments, Expenses and Reports, with appropriate tables, forms, filters, actions and summary information. Maintain the same visual system across all modules and make the interface feel like one cohesive accounting product rather than a collection of separate pages.

Prompt 4 : Responsive & Visual Refinement

Refine the SmartBook interface for a polished production-ready experience across desktop, tablet and smaller screens. Maintain the existing white and gray visual system, blue accent, professional typography and clean spacing. Ensure the sidebar, dashboard cards, tables, forms, charts and accounting modules adapt properly to different screen sizes. Avoid gradients, glassmorphism, neon colors and unnecessary visual effects.



08. Build: Prototype Mode

Objective

Convert the approved Design Mode prototype into a functional React application without changing the existing UI.

Implementation

Build was used to create:

- Dashboard
- Customers
- Vendors
- Invoices
- Expenses
- Payments
- Reports
- Settings
- Forms and reusable components
- Navigation and responsive layouts

The Build prompt explicitly instructed the implementation to preserve the approved design and focus on functionality and code quality.

Prompts Used

Prompt 1 : Convert Finalized Design into Application

The SmartBook UI design has been finalized. Do not redesign or change the existing layout, colors, typography, spacing, visual hierarchy or components. Now convert the approved design into a functional React.js application. Implement the existing Dashboard, Customers, Vendors, Invoices, Payments, Expenses, Reports and Settings pages using reusable components, proper navigation and responsive layouts. Keep the approved SmartBook design as the source of truth throughout the implementation.

Prompt 2 : Accounting Functionality

Continue developing the existing SmartBook application without changing the approved UI. Implement the core accounting workflows for customers, vendors, invoices, payments and expenses. Add the required forms, tables, actions, validation and state handling so users can create, view, update and manage accounting records through the application. Ensure the different modules remain connected and the overall experience feels like one complete accounting management system.

Prompt 3 : Backend & Database Integration

The SmartBook frontend and UI are already implemented. Now connect the application with the backend and MongoDB database while keeping the existing frontend design unchanged. Set up the required API communication and database operations for users, customers, vendors, invoices, invoice items, expenses and payments. Replace static or mock data with persistent application data and make sure the main accounting workflows correctly save and retrieve records from the database.

Prompt 4 : Functional Iteration & Verification

Review the current SmartBook implementation and verify the major application workflows end-to-end. Check dashboard data, customer and vendor records, invoice creation, payment tracking, expense management, forms, navigation and responsive behavior. Identify incomplete functionality or integration issues and implement the required corrections without changing the approved design. Re-test the affected workflows after each change and ensure the application remains stable.

09. Build Prompt Highlights

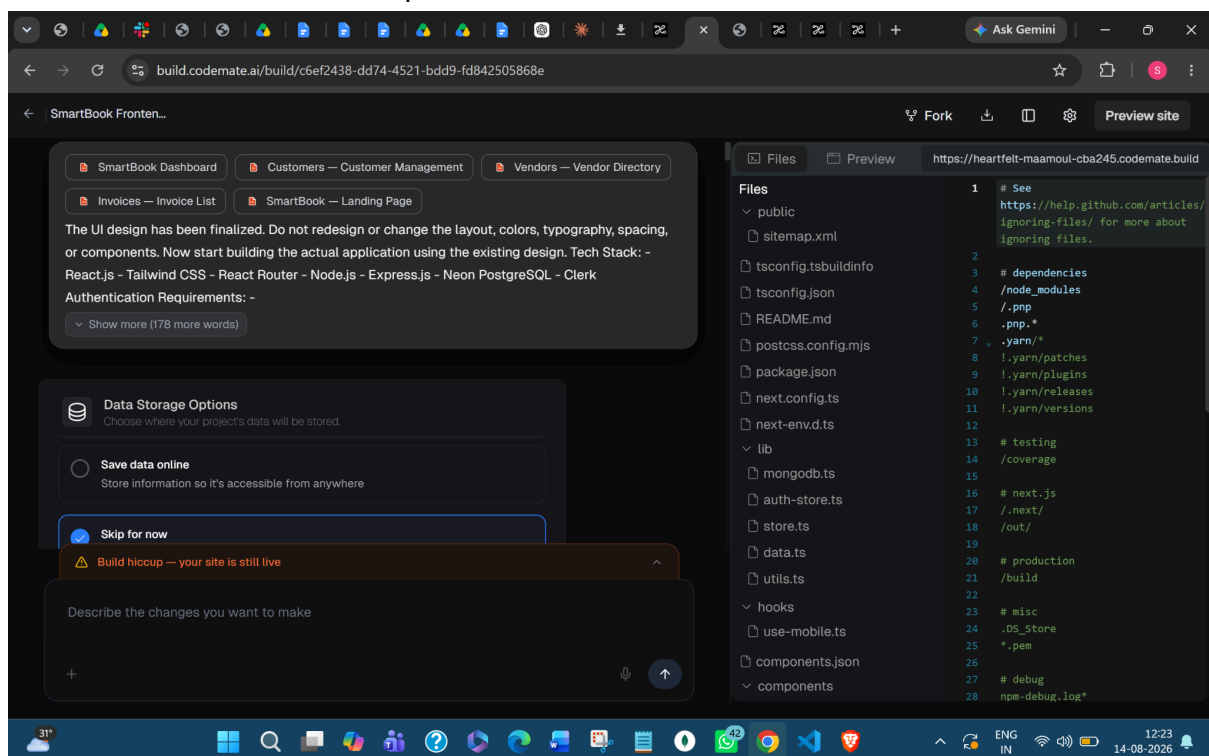
Design Mode

Design a modern, production-ready web application UI for an accounting software called SmartBook. The prompt then defined the complete SmartBook visual system, dashboard, sidebar, charts and accounting modules.

Prototype Mode

The UI design has been finalized. Do not redesign or change the layout, colors, typography, spacing, or components. Now start building the actual application using the existing design.

The prompt then defined the technology stack, reusable components, navigation, pages, forms, tables and validation requirements.



Prompts Used

Prompt 1 : Product Design

Design a modern, production-ready web application UI for an accounting software called "SmartBook". Create a clean, professional and responsive SaaS experience with a white background, subtle gray surfaces, blue primary accent, spacious layouts and an enterprise-style visual language suitable for accounting software.

Prompt 2 : Dashboard

Create a SmartBook accounting dashboard that gives users an immediate overview of their business finances. Include total revenue, total expenses, pending payments, paid invoices and active customers, along with monthly revenue versus expenses, expense categories, recent transactions and recent activity. Keep the dashboard clean, readable and focused on useful financial information.

Prompt 3 : Accounting Management

Build the core SmartBook accounting modules for Customers, Vendors, Invoices, Payments, Expenses and Reports. Each module should provide the appropriate tables, forms, actions and information required to manage accounting records efficiently. Maintain consistent navigation, components, spacing and styling across the entire application.

Prompt 4 : Functional Application

Take the finalized SmartBook design and turn it into a complete functional accounting application. Preserve the existing UI while implementing reusable React components, navigation, forms, tables, accounting workflows, validation and responsive behavior. The final application should feel like a production-ready accounting product rather than a static prototype.

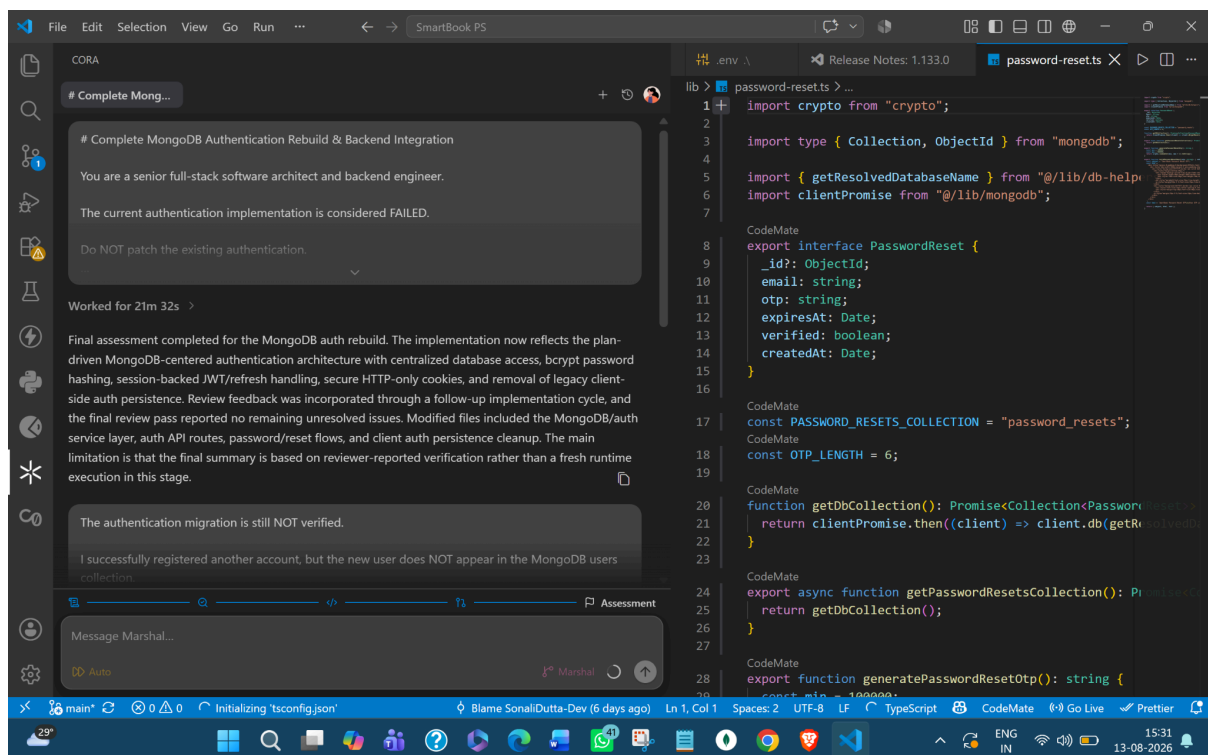
10. CORA: Planning & Implementation

CORA was used for the backend and authentication implementation, particularly to rebuild the authentication layer and establish proper database connectivity.

Main Objective

- Rebuild authentication
- Connect backend with MongoDB
- Remove broken authentication logic
- Implement secure authentication
- Verify the complete runtime flow

The documented CORA task specifically required a complete authentication rebuild rather than patching the existing implementation.



Prompts Used

Prompt 1 : Authentication Rebuild

Analyze the current SmartBook authentication implementation and create a structured plan to rebuild the authentication layer properly. Review the existing login, registration, logout, session handling, middleware and authentication-related files. Identify the required backend routes, user data handling, authentication logic and dependencies, and define the implementation

steps needed to establish a reliable authentication flow without leaving broken or duplicated authentication logic.

Prompt 2 : MongoDB Authentication Integration

Rework the SmartBook authentication system so that it is properly connected to MongoDB and works reliably at runtime. Review the existing database connection, user model, authentication routes and session handling before making changes. Implement the required database operations, password handling, JWT-based authentication and protected route logic while keeping the existing application structure intact. Verify that authentication data is correctly persisted and retrieved from MongoDB.

Prompt 3 : Authentication Runtime Verification

Review the implemented SmartBook authentication flow end-to-end and verify registration, login, logout, session validation and protected routes. Check the communication between the frontend authentication pages, API routes, middleware and MongoDB. Identify any runtime errors, incorrect session behavior or database connectivity issues, then implement the necessary corrections and re-test the complete authentication flow.

Prompt 4 : Final Authentication & Application Integration

Verify that the rebuilt SmartBook authentication layer integrates correctly with the rest of the application. Ensure authenticated users can successfully move through the login flow and access the appropriate application pages while unauthenticated requests remain protected. Check database connectivity, session persistence, API responses, navigation after authentication and logout behavior. Make the required implementation refinements and verify the complete flow after the changes.

11. CORA Task Archive

Authentication

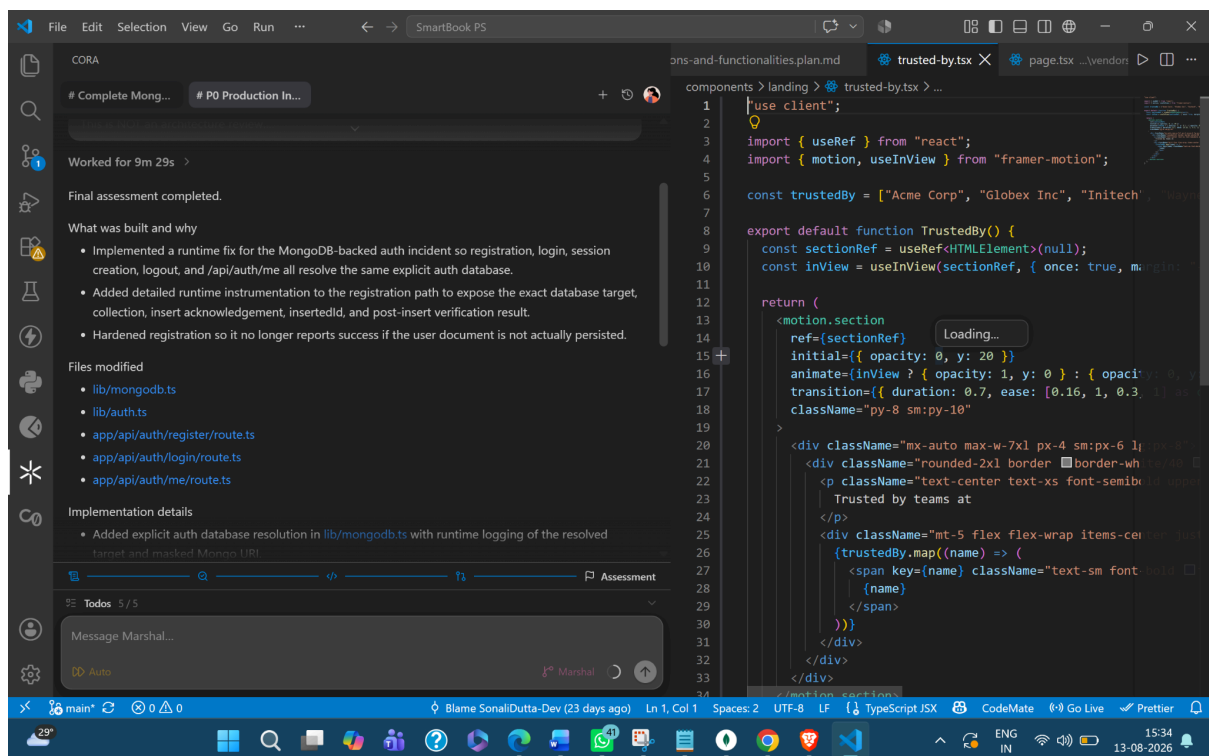
Implemented and verified:

- Registration
- Login
- Logout
- JWT authentication
- Password hashing
- Refresh tokens
- Protected routes
- Session validation

Database

CORA handled:

- MongoDB connection
- User collection
- Session collection
- Environment validation
- Runtime database verification



12. Implementation Milestones

01. Design

Created the initial SmartBook UI using Build Design Mode.

02. Frontend

Converted the approved design into a functional React application using Build.

03. Backend

Integrated backend functionality and database operations.

04. Authentication

Used CORA to rebuild and verify the authentication architecture.

05. Verification

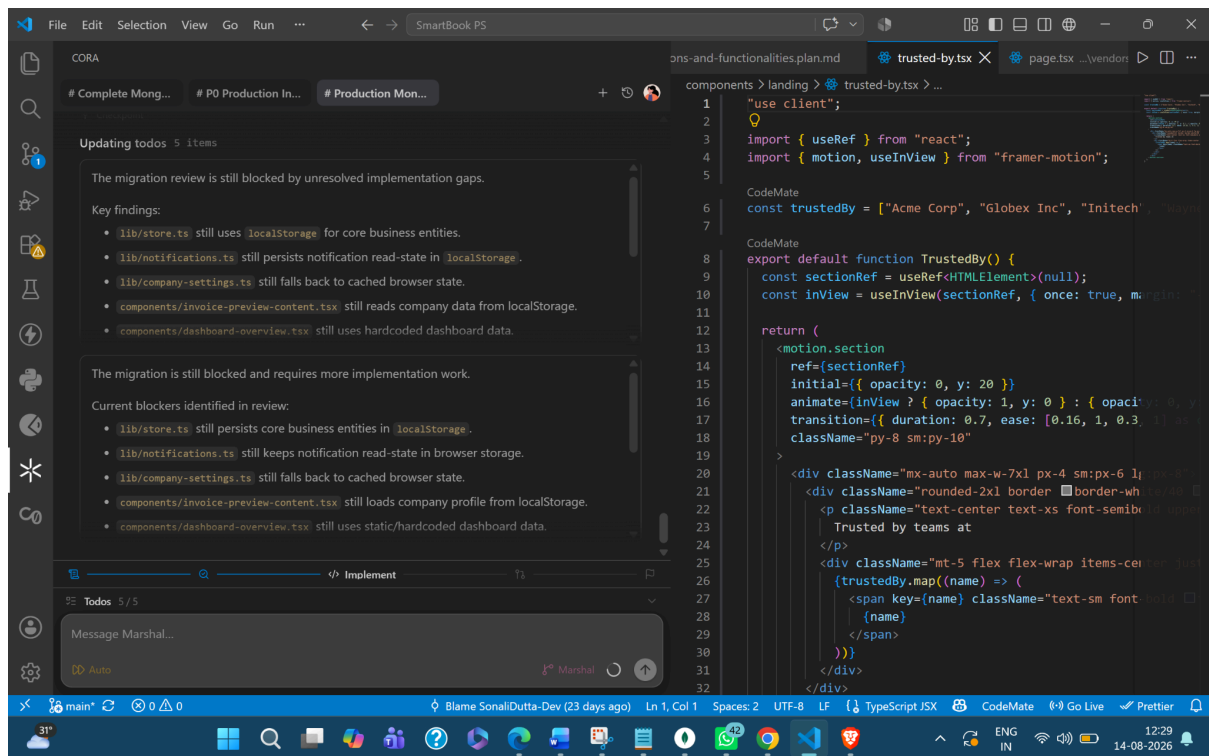
Tested registration, login, sessions, logout and database connectivity.

13. Backend, Data & Integrations

The SmartBook data model includes:

- Users
- Companies
- Customers
- Vendors
- Invoices
- Invoice Items
- Expenses
- Payments

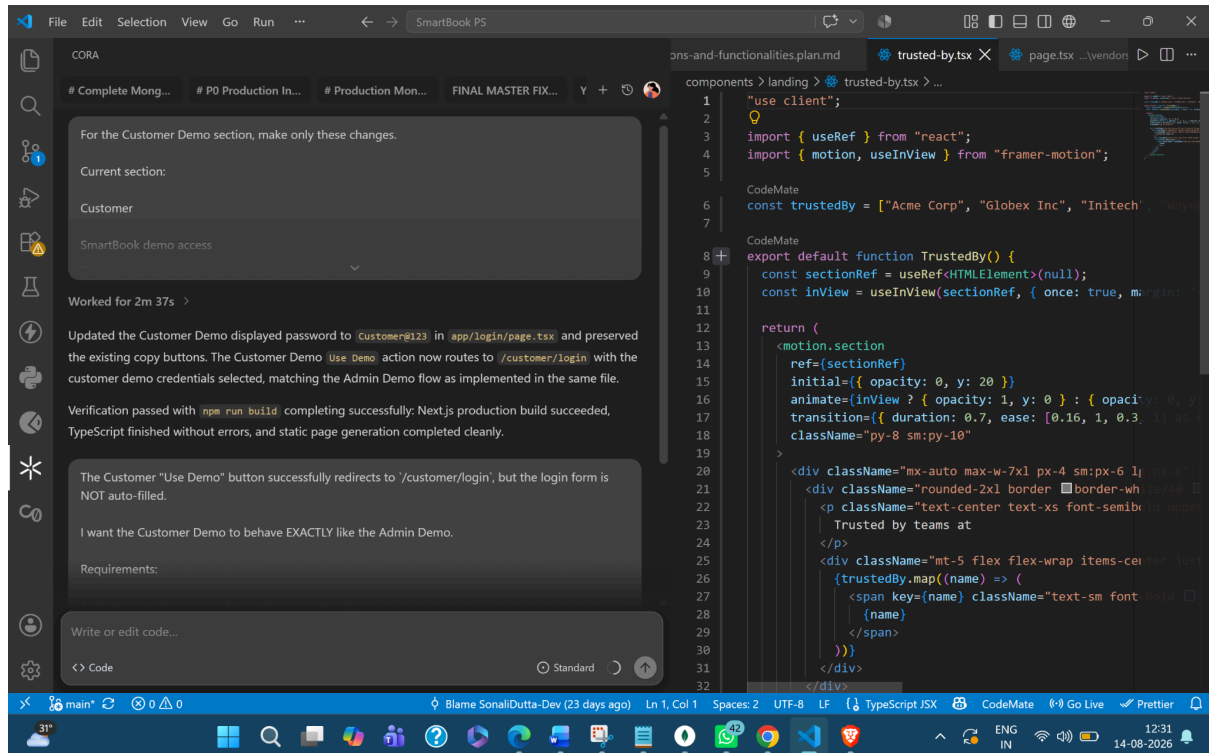
The backend connects the application workflows with the database and handles the application's accounting operations.



14. Authentication & User Management

SmartBook uses **Clerk Authentication** as defined in the project stack, with role-based access for administrators and staff/accountants.

The authentication implementation was also verified through registration, login, session handling and logout flows.



15. Responsive UI & Polish

The UI was designed as a responsive, desktop-first accounting application.

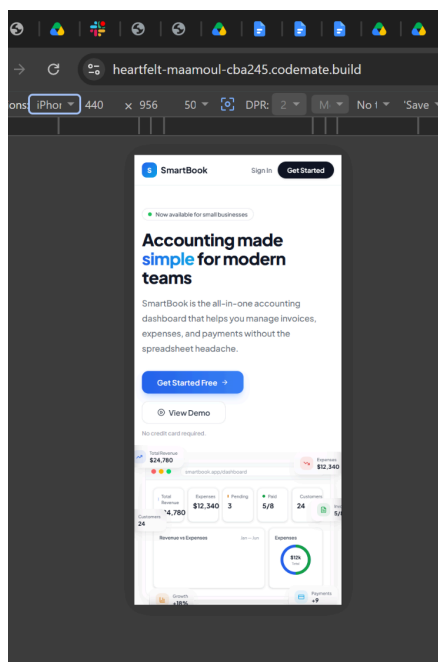
The interface uses:

- Clean layouts
- Responsive components
- Consistent spacing
- Professional typography
- Minimal icons
- Reusable UI components

The Design Mode specification explicitly required a responsive desktop-first interface with clean spacing and professional styling.

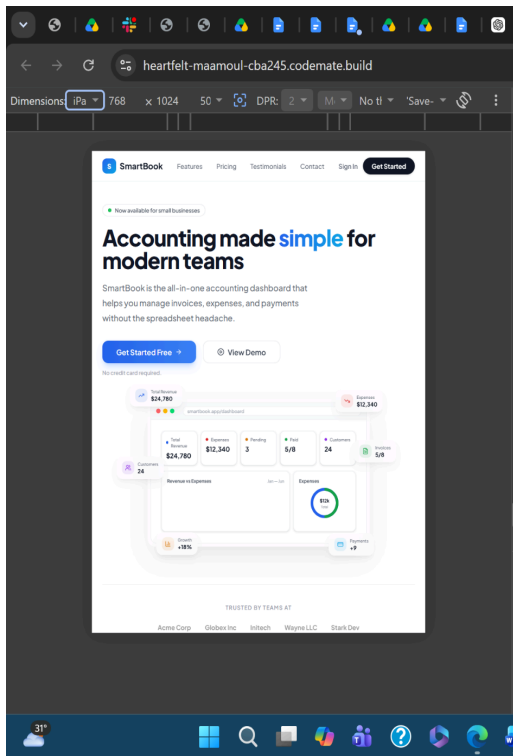
Mobile

Collapsed navigation, stacked layouts and touch-friendly interactions.



Tablet

Adjusted navigation and content density.



Desktop

Full sidebar, dashboards, tables and multi-column layouts.

